

REPORTE DE CIERRE

Tickets 4 y 6

Colas durables, control humano, registro outbound y evidencia real en GHL Sandbox / Render

Resultado ejecutivo

La implementación solicitada quedó desplegada y validada en producción para la parte técnica. Las pruebas locales, Redis, PostgreSQL/RLS, registro outbound y eventos firmados de GHL tienen evidencia. La única salvedad es que no fue posible generar un nuevo inbound real porque el contacto Sandbox no tiene teléfono ni canal de entrada habilitado.

Area	Resultado	Lectura sencilla
Ticket 4 - colas Redis	Validado	Los trabajos no se borran antes de terminar; se pueden reclamar, reintentar y enviar a DLQ.
Ticket 6 - takeover humano	Validado	El control humano pausa la automatización y la conversación queda bajo responsabilidad del equipo.
RLS/FORCE RLS real	Aprobado	Un tenant no puede ver los datos de otro tenant.
GHL Sandbox firmado	Capturado	Se recibieron eventos reales de quitar/poner tag y comentario interno.
Inbound real posterior	Pendiente por limitación del Sandbox	No se simuló ni se presentó como evidencia real.

Fecha de corte: 20 de agosto de 2026

Repositorio: pixelmediacolombia-sudo/1987-bdc-os, rama master

Despliegue verificado: commit eee0c87 - fix(webhooks): normalize missing GHL channel

1. Que se hizo y por que importa

Esta sección resume cada solicitud con una explicación no técnica. El objetivo es que el equipo pueda entender qué quedó protegido, qué se probó y qué no debe interpretarse como probado.

Solicitud	Que se hizo	Explicación no técnica	Estado
Aplicar RLS y FORCE RLS a outbound_message_registry	Se activó aislamiento por tenant y se ejecutó la matriz cross-tenant en PostgreSQL real.	Cada negocio queda encerrado en su propio espacio; una consulta sin contexto correcto no devuelve datos de otro negocio.	APROBADO
Ejecutar consultas bajo tenant real	Las consultas del registro y webhooks establecen app.tenant_id antes de leer o escribir.	El sistema primero identifica de qué negocio viene el evento y luego trabaja solo dentro de ese negocio.	APROBADO
Registrar outbound antes del proveedor	RegisteredOutboundMessageSender reserva el mensaje antes de invocar al proveedor y adjunta el ID después del éxito.	Si el proveedor confirma un mensaje, el sistema sabe si lo originó 1987 y no lo confunde con una respuesta humana.	APROBADO
No hacer ZREM antes del burst job	La cola reclama el trabajo y solo confirma después del éxito; ante error lo reprograma.	Un trabajo que falla no desaparece: vuelve a intentarse.	APROBADO
Backoff, máximo de intentos y DLQ	RedisWebhookQueue aplica espera creciente, límite de intentos y cola de mensajes muertos.	Los errores temporales tienen otra oportunidad; los errores persistentes quedan separados para revisión.	APROBADO
Pruebas automáticas	Se agregaron pruebas de colas, registro outbound, RLS y aislamiento cross-tenant.	Las reglas importantes quedan comprobadas automáticamente y no dependen solo de una revisión manual.	APROBADO
Quitar y volver a poner human_takeover	Se hizo en el mismo contacto del Sandbox y ambos eventos quedaron procesados.	Se comprobó que quitar y volver a activar el control son dos eventos independientes.	APROBADO
Redis real dentro de Render	Las pruebas de integración se ejecutaron usando la infraestructura Redis interna de Render.	Se probó con el Redis que usará el servicio desplegado, no únicamente con un sustituto local.	VALIDADO
Payloads firmados reales de GHL	Se capturaron ContactTagUpdate y OutboundMessage con firma presente y estado procesado.	Se comprobó el camino real desde GHL hasta el backend, incluyendo la firma de autenticidad.	APROBADO
Takeover -> pausa -> inbound -> cero IA	Takeover y pausa quedaron demostrados en producción; el inbound real posterior no pudo generarse.	La protección está activa y las pruebas locales demuestran que una conversación pausada no reactiva IA, pero falta el mensaje real posterior del contacto.	SALVEDAD

2. Validacion local

Se ejecutaron compilación, revisión de tipos y la suite de seguridad/control. El resultado fue 27 pruebas aprobadas, 0 fallos y 3 omitidas porque requieren servicios reales externos o variables de integración.

Comprobación	Resultado	Qué demuestra
npm run typecheck	PASS	El código mantiene coherencia de tipos.
npm run build	PASS	El servicio puede compilarse para Render.
npm run test:security	27 pass / 0 fail / 3 skipped	Las reglas de firma, parser, takeover, colas, registro y memoria pasan automáticamente.

Pruebas destacadas que pasaron

- La eliminación y reapiación de human_takeover son eventos independientes del mismo contacto.
- Una conversación pausada no reactiva IA con un nuevo inbound.
- Takeover pausa, el inbound queda consumido sin downstream y el orquestador no se ejecuta.
- El registro outbound establece contexto de tenant antes de cada consulta.
- El sender reserva antes del proveedor y adjunta el ID solo después del éxito.
- La burst queue reclama primero, confirma después del éxito y reprograma ante fallo.
- La webhook queue aplica backoff exponencial y mueve fallos terminales a DLQ.

Estado de mensajería e IA

El despliegue mantiene el orquestador outbound desactivado. El componente activo solo hidrata contexto y, si la conversación está pausada, termina sin invocar proveedor ni acción externa. El único mensaje real enviado durante la evidencia fue un comentario interno manual en GHL, visible solo para el equipo.

3. PostgreSQL real y aislamiento por tenant

La prueba se ejecutó contra la base PostgreSQL de producción desde el Shell de Render. Se validó que `outbound_message_registry` tiene RLS y `FORCE RLS`, y que la matriz de aislamiento impide cruzar datos entre tenants.

```
{
  "db": "bdc_os_prod",
  "function": "resolve_ghl_tenant_id(text)",
  "outbound_registry_rls": true,
  "outbound_registry_force_rls": true,
  "tenant_routes": 1,
  "matrix_test": {
    "name": "RLS cross-tenant matrix isolates integrations, audits, raw webhooks, and outbound
registry",
    "tests": 1,
    "pass": 1,
    "fail": 0
  }
}
```

Explicación no técnica

RLS es una barrera dentro de la base de datos. `FORCE RLS` hace que esa barrera también aplique a los caminos privilegiados del servicio. La prueba cross-tenant intentó leer datos con el contexto equivocado y confirmó que no se filtran registros de otro tenant.

También se comprobó que las consultas del registro outbound se ejecutan después de establecer el tenant real. No se imprimieron URLs, tokens, contraseñas ni valores de entorno.

4. Evidencia real de GHL Sandbox

Se usó el contacto real del GHL Sandbox: Prueba Prueba. No es un fixture local ni un contacto productivo. Su identificador de contacto fue s0kF500JnN8vUh4TrHoi y la ubicación fue C89276AhU942sfwFMQ9m.

4.1 Quitar y volver a aplicar human_takeover

Primero se quitó el tag; luego se volvió a aplicar al mismo contacto. Ambos webhooks llegaron con firma y quedaron procesados.

```
{
  "remove": {
    "event_type": "ContactTagUpdate",
    "external_id": "b925646b-d5b2-4d5a-b327-893c09bd123b",
    "location_id": "C89276AhU942sfwFMQ9m",
    "contact_id": "s0kF500JnN8vUh4TrHoi",
    "tags": [],
    "signature_present": true,
    "backend_status": "processed",
    "received_at": "2026-08-20T19:09:26.625Z"
  },
  "reapply": {
    "event_type": "ContactTagUpdate",
    "external_id": "9189858e-6045-4a20-9cdb-ab733cb0ee86",
    "location_id": "C89276AhU942sfwFMQ9m",
    "contact_id": "s0kF500JnN8vUh4TrHoi",
    "tags": [
      "human_takeover"
    ],
    "signature_present": true,
    "backend_status": "processed",
    "received_at": "2026-08-20T19:09:43.592Z"
  }
}
```

Explicación no técnica

El primer registro dice que el contacto quedó sin el tag de control. El segundo dice que el mismo contacto volvió a tener human_takeover. Esto demuestra que el sistema reconoce correctamente el cambio de estado en ambos sentidos.

4.2 Payload firmado real de OutboundMessage

Se creó un comentario interno autorizado en GHL. GHL lo reportó como delivered y el backend lo procesó después del ajuste que normaliza la ausencia de canal en InternalComment.

```
{
  "event_type": "OutboundMessage",
  "external_id": "18b1abbf-21f5-458e-84c5-cd4f9ac89f5f",
  "location_id": "C89276AhU942sfwFMQ9m",
  "contact_id": "s0kF500JnN8vUh4TrHoi",
  "signature_present": true,
  "backend_status": "processed",
  "error_message": null,
  "received_at": "2026-08-20T19:19:57.718Z",
  "payload": {
    "body": "Sandbox outbound evidence after parser fix: human_takeover active.",
    "type": "OutboundMessage",
    "appId": "6a84ce51032e61130732e5cd",
    "source": "app",
    "status": "delivered",
    "userId": "ZedHfRl4eUkLH3C62rFv",
    "contactId": "s0kF500JnN8vUh4TrHoi",
    "dateAdded": "2026-08-20T19:19:56.410Z",
    "direction": "outbound",
    "messageId": "6I17o351g3WNgXc0xMVN",
    "timestamp": "2026-08-20T19:19:57.145Z",
    "versionId": "6a84ce51032e61130732e5cd",
    "webhookId": "18b1abbf-21f5-458e-84c5-cd4f9ac89f5f",
    "locationId": "C89276AhU942sfwFMQ9m",
    "attachments": [],
    "contentType": "text/plain",
    "messageType": "InternalComment",
    "messageTypeId": 37,
    "conversationId": "UfbTy7KnAfWNSusRsapg",
    "messageTypeString": "TYPE_INTERNAL_COMMENT"
  }
}
```

Nota de seguridad: el campo signature se conserva en PostgreSQL y se verificó que está presente. El valor criptográfico completo no se reproduce en este informe para evitar convertir el reporte en un contenedor de material de autenticación.

5. Takeover y pausa real

Después del comentario interno, PostgreSQL mostró la conversación real en pausa y bajo responsabilidad del equipo humano. También registró la decisión de cancelar la ejecución de IA.

```
{
  "conversation": {
    "ghl_conversation_id": "UfbTy7KnAfWNSusRsapg",
    "contact_id": "s0kF500JnN8vUh4TrHoi",
    "channel": "other",
    "state": "paused",
    "owner": "staff",
    "last_activity": "2026-08-20T19:19:57.718Z"
  },
  "decision_log": {
    "selected_action": "pause_ai",
    "reason": "AI execution cancelled: Human operator took control (Trigger: Staff Message)",
    "created_at": "2026-08-20T19:19:57.718Z"
  }
}
```

Explicación no técnica

La conversación quedó en estado paused y owner staff. En términos prácticos, el sistema reconoce que una persona tomó el control y no debe responder automáticamente. El registro pause_ai deja una huella auditable de esa decisión.

Salvedad sobre el inbound real

El contacto Sandbox no tenía número telefónico ni otro canal habilitado para que entrara un mensaje real después de la pausa. Por eso no se fabricó un webhook ni se usó un evento sintético para declarar cerrada esa parte. La prueba local equivalente sí pasó: una conversación paused recibe un nuevo inbound y el orquestador registra cero activaciones de IA.

Qué falta para cerrar la evidencia E2E completa

Solo hace falta que el mismo contacto reciba un mensaje entrante real por un canal habilitado, después de human_takeover. El resultado esperado es: evento InboundMessage procesado, conversación todavía paused y cero invocaciones de IA/proveedor.

6. Colas Redis y registro outbound

Las colas fueron ajustadas para conservar trabajos hasta que el procesamiento termine. La confirmación solo ocurre después del éxito. Los errores se reintentan con espera creciente y, al superar el límite, se trasladan a una dead-letter queue para inspección.

Componente	Regla comprobada	Explicación sencilla
Burst buffer	Claim antes de procesar; ACK posterior al éxito; reprogramación ante error.	Una tarea no desaparece porque el proceso se haya caído a mitad de camino.
RedisWebhookQueue	Backoff exponencial, máximo de intentos y DLQ.	Los fallos temporales se reintentan; los persistentes quedan apartados.
Outbound registry	Reserva antes del proveedor y asociación del provider_message_id después.	Se puede distinguir una respuesta automática propia de una respuesta de un humano.

Pruebas automáticas relacionadas

Los nombres de las pruebas que pasaron fueron: burst queue claims first, ACKs after success, and reprograms a failure; webhook queue applies exponential backoff and moves terminal failures to DLQ; outbound registry sets tenant context before every query; registered sender reserves before provider and attaches provider id after success.

Redis real en Render

La infraestructura de Redis se probó desde el servicio desplegado en Render usando la URL interna configurada en producción. Esto es diferente de las pruebas locales con dobles de memoria: valida la conexión y el comportamiento en el entorno operativo previsto.

7. Matriz final de evidencia

Evidencia	Dónde	Resultado	Observación
Compilación y tipos	Local	PASS	Sin errores.
Suite automática	Local	27 pass / 0 fail / 3 skipped	Los omitidos requieren integración externa.
RLS/FORCE RLS	PostgreSQL real en Render	PASS	Matriz cross-tenant aprobada.
Redis	Redis interno de Render	VALIDADO	Colas durables, reintentos y DLQ comprobados.
ContactTagUpdate quitar	GHL Sandbox real	processed	Firma presente; tags vacíos.
ContactTagUpdate reaplicar	GHL Sandbox real	processed	Firma presente; human_takeover presente.
OutboundMessage	GHL Sandbox real	processed	InternalComment; GHL delivered; no SMS.
Conversation pause	PostgreSQL real	paused	Owner staff; decision pause_ai.
Nuevo inbound real posterior	GHL Sandbox	NO EJECUTADO	Contacto sin teléfono/canal de entrada.

Conclusión para tickets 4 y 6

La implementación técnica y la evidencia de seguridad/operación están completas para el alcance probado. Puede cerrarse la parte de colas, persistencia, aislamiento, takeover y payloads reales con la salvedad explícita de que el último paso E2E - un inbound real después de la pausa - queda pendiente por una limitación del contacto Sandbox, no por un fallo observado del control de pausa.

Estado de seguridad

Mensajería automática e IA deben permanecer desactivadas hasta que se obtenga ese último inbound real y se confirme en producción el resultado esperado. El reporte no autoriza activar envíos ni IA.

8. Commits y despliegues realizados

Estos commits se hicieron para que cada cambio quedara separado, revisable y recuperable: primero se endurecieron las colas y el registro; después se estabilizaron las pruebas con Redis y PostgreSQL reales; finalmente se corrigió el caso descubierto por el payload real de GHL. Todos fueron enviados a master y Render hizo el despliegue automático correspondiente.

Commit	Nombre y URL	Por qué se hizo	Resultado
2b41ed0	feat(control): harden durable queues and tenant registry https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/2b41ed0	Base de colas durables, contexto de tenant y registro outbound.	Controles base implementados.
8ecbbc6	test(control): tolerate render redis latency https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/8ecbbc6	La latencia real de Render podía hacer frágiles las pruebas.	Pruebas más estables.
a009c07	test(redis): stabilize production integration timing https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/a009c07	Ajustar tiempos de integración contra Redis real.	Integración Redis estabilizada.
dbf5808	test(redis): assert durable burst handoff before worker https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/dbf5808	Demostrar que el burst queda durable antes del worker.	Entrega durable comprobada.
fa44b92	test(rls): use preprovisioned tenant context https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/fa44b92	La matriz necesita un tenant real y controlado.	RLS probado con contexto válido.
0a27e66	test(rls): isolate production integration fixture https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/0a27e66	Evitar que una fixture cruce datos de producción.	Aislamiento reforzado.
ca18492	test(rls): reuse real GHL integration fixture https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/ca18492	Usar la integración GHL real para no depender de datos inventados.	Matriz real ejecutada.
eee0c87	fix(webhooks): normalize missing GHL channel https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/eee0c87	El payload real InternalComment no traía canal y fallaba la persistencia.	Repetición procesada y conversación pausada.

URLs directas de los commits

<https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/2b41ed0>

<https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/8ecbbc6>

<https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/a009c07>

<https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/dbf5808>

<https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/fa44b92>

<https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/0a27e66>

<https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/ca18492>

<https://github.com/pixelmediacolombiasudo/1987-bdc-os/commit/eee0c87>

Despliegue final verificado en Render

Render registró: Deploy live for eee0c87: fix(webhooks): normalize missing GHL channel, el 20 de agosto de 2026 a las 2:18 PM. El ajuste se verificó repitiendo el comentario interno real y consultando PostgreSQL.

Qué cambió con el último commit

El primer comentario interno real llegó firmado desde GHL, pero quedó failed porque el payload no traía canal y el valor unknown no estaba permitido por la base de datos. El commit eee0c87 normalizó ese caso a other. Al repetir la prueba, el siguiente OutboundMessage quedó processed y la conversación quedó paused.

Fin del reporte

Actualizacion posterior: limpieza de codigo

Se incorpore esta pagina despues de una revision adicional solicitada para dejar el codigo sin declaraciones no utilizadas ni archivos temporales de la revision.

Que se reviso

Se ejecuto TypeScript con noUnusedLocals y noUnusedParameters activados. La primera pasada encontro cuatro elementos sin uso: un tipo PoolClient en una prueba, un parametro eventType que no se utilizaba, la funcion isStaffSender sin referencias y el tipo WebhookQueueJob sin referencias.

Que se hizo

Se eliminaron esos elementos porque no participaban en el comportamiento del sistema. No se comento codigo muerto ni se dejaron variables reservadas sin uso: la base queda limpia y el compilador estricto queda en verde.

Elemento	Accion	Motivo
PoolClient	Eliminado del import de la prueba	No se usaba en el fake pool.
eventType en buildMessage	Eliminado del parametro	La funcion no lo leia.
isStaffSender	Funcion eliminada	No tenia ninguna llamada.
WebhookQueueJob	Eliminado del import	La cola usa el handler directamente.

Validacion despues de la limpieza

noUnusedLocals: PASS. noUnusedParameters: PASS. Suite de seguridad: 27 pass, 0 fail, 3 skipped por requerir integraciones externas. Diff check: PASS.

Commit publicado en GitHub

2bbcb19 - chore(cleanup): remove unused declarations

<https://github.com/pixelmediacolombia-sudo/1987-bdc-os/commit/2bbcb19>

El commit se publico en master y Render queda listo para desplegarlo automaticamente.